

INTERVIEW PREPARATION

Database Concepts

AI Engineer ke liye — Roman Urdu mein Asaan
Explanation

SQL & RDBMS • Joins • Normalization • ACID & Transactions

Indexing • NoSQL (Document, Key-Value, Column, Graph)

Vector Databases • Embeddings & RAG • Feature Store • Caching & Scaling

Goal: Interview ke liye database concept clear karna, especially AI/ML systems ke liye

Intro — Ye Guide Kaise Use Karein

Assalam-o-Alaikum! Ye notes specially AI Engineer role ke interviews ke liye banayi gayi hain. Database questions do tarah se puchay ja saktay hain:

- **Traditional DB concepts** — SQL, joins, normalization, indexing, ACID, transactions (ye har software/AI role mein puchay jate hain, kyunke AI systems ka data kahin na kahin database mein hi store hota hai).
- **AI-specific DB concepts** — Vector Databases, embeddings storage, similarity search, RAG pipelines, feature stores (ye specifically AI Engineer roles mein bohat important ho gaye hain, kyunke LLM applications aur RAG systems inhi par based hotay hain).

Tip: AI Engineer interviews mein aksar dono mix hotay hain — "SQL query likh kar dikhao" aur "RAG system mein vector database kaise use hota hai" dono puchay ja saktay hain. Is liye ye guide dono cover karti hai.

Contents

- 1 Database Basics — RDBMS vs NoSQL
- 2 SQL — Practical Queries (SELECT, JOIN, GROUP BY, Subqueries)
- 3 Normalization & Database Design
- 4 ACID Properties & Transactions
- 5 Indexing — Kaise Kaam Karta Hai
- 6 NoSQL Databases — Types & Use Cases
- 7 Vector Databases — AI Engineer ka Sab Se Important Topic
- 8 Embeddings & Similarity Search — Concept
- 9 RAG (Retrieval Augmented Generation) mein Database ka Role
- 10 Feature Store — ML Systems ke liye
- 11 Caching (Redis) & Scaling (Sharding/Replication)
- 12 Bonus: Interview Questions & Cheat Sheet

1. Database Basics — RDBMS vs NoSQL

THEORY

Database ek organized system hai jahan data store, retrieve, aur manage kiya jata hai. Do main categories hain:

Relational Database (RDBMS)

Data **tables** (rows aur columns) ki form mein store hota hai, aur tables ke beech **relationships** (foreign keys) define ki jati hain. Har table ka ek fix **schema** hota hai (matlab columns aur unke data types pehle se decide hotay hain).

Examples: MySQL, PostgreSQL, SQLite, Microsoft SQL Server.

NoSQL Database

Data ka **fix schema** zaroori nahi, aur relational tables ki bajaye alag alag formats mein store hota hai (document, key-value, graph, waghera). Flexible aur horizontally scalable hone ki wajah se bare, unstructured, ya fast-changing data ke liye best hai.

Examples: MongoDB (document), Redis (key-value), Cassandra (column), Neo4j (graph), Pinecone/Weaviate (vector).

SQL vs NoSQL — Interview ka Favorite Comparison

Aspect	SQL (RDBMS)	NoSQL
Schema	Fixed, pehle se define	Flexible / Dynamic
Data Structure	Tables (rows/columns)	Documents, Key-Value, Graph, waghera
Scaling	Vertical (bara server)	Horizontal (zyada servers)
Relationships	Strong (foreign keys, joins)	Weak ya manual
Best For	Structured data, transactions (banking, e-commerce orders)	Unstructured/semi-structured data, big data, real-time apps
Consistency	Strong (ACID)	Aksar "eventual consistency" (BASE model)

AI Engineer Angle: ML pipelines mein aksar dono use hotay hain — structured metadata (user info, model logs) SQL mein, jabke embeddings/unstructured text/images NoSQL ya Vector DB mein store hotay hain.

2. SQL — Practical Queries

PRACTICAL

Ye sab se zyada puchay jane wale SQL patterns hain. Example ke liye maan lein humare paas do tables hain:

```
-- employees table
-- id | name   | department_id | salary
-- 1  | Ali    | 10            | 80000
-- 2  | Sara   | 20            | 95000
-- 3  | Bilal  | 10            | 70000

-- departments table
-- id | dept_name
-- 10 | Engineering
-- 20 | Data Science
```

Q1. Basic SELECT + WHERE + ORDER BY

```
SELECT name, salary
FROM employees
WHERE salary > 75000
ORDER BY salary DESC;
```

Q2. JOIN — Do Tables ka Data Combine Karna

INNER JOIN: Sirf wo rows return karta hai jinka match dono tables mein ho.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;
```

LEFT JOIN: Left table ke saare rows return karta hai, chahe right table mein match ho ya na ho (na milne par NULL).

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;
```

Interview Tip: INNER vs LEFT vs RIGHT vs FULL OUTER JOIN ka farq zaroor yaad rakhein. INNER = sirf matching, LEFT = left table poori + matching right, RIGHT = right table poori + matching left, FULL OUTER = dono tables poori (matching + non-matching, jahan match na ho wahan NULL).

Q3. GROUP BY + Aggregate Functions

Har department ka average salary nikalna:

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary, COUNT(*) AS total_employees
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.dept_name;
```

Q4. HAVING — Group Karne ke Baad Filter

WHERE group hone se pehle filter karta hai, **HAVING** group hone ke baad. Jaise: sirf un departments dikhao jinki average salary 80000 se zyada ho:

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 80000;
```

Q5. Subquery — Query ke Andar Query

Sab se zyada salary lene wale employee ka naam nikalna:

```
SELECT name
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Q6. Second Highest Salary Dhoondna (Bohat Common Interview Question)

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Ya LIMIT/OFFSET se (PostgreSQL/MySQL):
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Q7. Window Functions (Advanced lekin AI roles mein bhi puchay jate hain)

Har department mein employees ko salary ke hisab se rank dena:

```
SELECT name, department_id, salary,
       RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rank
FROM employees;
```

ROW_NUMBER() vs RANK() vs DENSE_RANK(): ROW_NUMBER hamesha unique number deta hai (1,2,3...). RANK ties ko same number deta hai lekin agla number skip karta hai (1,1,3). DENSE_RANK ties ko same number deta hai bina skip kiye (1,1,2).

General Rule: Proper indexes ke sath simple SELECT/WHERE $O(\log n)$ tak fast ho sakta hai, lekin bina index ke full table scan $O(n)$ lagta hai — is liye large tables par indexing critical hai (Section 5 dekhein).

3. Normalization & Database Design

THEORY

Normalization ek process hai jisme data ko is tarah organize kiya jata hai ke **redundancy (duplicate data)** kam ho aur data integrity maintain rahe. Isko "Normal Forms" (1NF, 2NF, 3NF) ke through achieve kiya jata hai.

Normal Forms — Simple Explanation

- **1NF (First Normal Form):** Har column mein sirf ek value honi chahiye (atomic values), koi repeating groups nahi. Jaise agar ek column mein "Ali, Sara, Bilal" comma se likha ho, ye 1NF violate karta hai — inhein alag rows mein hona chahiye.
- **2NF (Second Normal Form):** 1NF ke sath sath, har non-key column पूरी primary key par depend karni chahiye (partial dependency na ho). Ye tab relevant hota hai jab composite (multiple column) primary key ho.
- **3NF (Third Normal Form):** 2NF ke sath sath, koi bhi non-key column kisi doosre non-key column par depend nahi karni chahiye (transitive dependency na ho). Jaise agar "employee" table mein "department_id" aur "department_name" dono hon, to department_name ko alag table mein le jana chahiye.

Normalization ka Faida

- Duplicate data kam hota hai → storage bachta hai.
- Data update karna asaan hota hai (ek jagah change karo, sab jagah reflect ho).
- Data inconsistency ka risk kam hota hai.

Denormalization — Ulta Process

Kabhi kabhi jaan-boojh kar redundancy add ki jati hai (denormalization) taake **read queries fast** ho jayein, kyunke bar bar joins karne ki zaroorat nahi parti. Ye tab use hota hai jab read-heavy system ho (jaise analytics/reporting dashboards, ya ML feature tables).

AI Engineer Angle: ML feature engineering mein aksar denormalized tables (wide tables) use hoti hain, kyunke training/inference ke waqt fast read chahiye hoti hai aur bar bar joins karna slow hota hai.

4. ACID Properties & Transactions

THEORY

Transaction database operations ka ek group hota hai jo **ek unit** ki tarah execute hota hai — ya to sab operations successful hongy, ya koi bhi nahi hoga. ACID ye guarantee deta hai ke transactions reliable tareeqe se process hon.

| ACID — Har Letter ka Matlab

- **A — Atomicity:** Transaction ke saare steps ya to poore complete hongay, ya bilkul bhi nahi (all or nothing). Jaise bank transfer mein: agar ek account se paisay katay aur doosre mein add na ho sakay, to poori transaction cancel/rollback ho jati hai.
- **C — Consistency:** Transaction ke baad database hamesha ek valid state mein hona chahiye (saare rules/constraints follow hon).
- **I — Isolation:** Multiple transactions ek sath chal rahi hon to wo ek doosre ko affect nahi karenge, jaise har transaction akeli chal rahi ho.
- **D — Durability:** Ek baar transaction commit ho jaye, to uska data permanent ho jata hai — chahe system crash bhi ho jaye, data lost nahi hoga.

Real-World Example (Bank Transfer)

```
BEGIN TRANSACTION;  
  
UPDATE accounts SET balance = balance - 500 WHERE id = 1; -- Ali se 500 katay  
UPDATE accounts SET balance = balance + 500 WHERE id = 2; -- Sara ko 500 milay  
  
COMMIT; -- dono steps successful, to permanent save  
-- agar beech mein error aye to: ROLLBACK; (sab kuch wapas undo ho jata)
```

Interview mein yaad rakhein: ACID = "All or nothing (Atomicity), Valid state (Consistency), Concurrent-safe (Isolation), Permanent (Durability)".

| BASE (NoSQL ka Alternative Model)

NoSQL databases aksar strict ACID ke bajaye **BASE** follow karte hain: **B**asically **A**vailable, **S**oft state, **E**ventual consistency — matlab system hamesha available rahega, lekin data turant consistent nahi hoga, thori der baad (eventually) sab replicas sync ho jayenge. Ye trade-off large-scale distributed systems mein speed aur availability ke liye kiya jata hai.

5. Indexing — Kaise Kaam Karta Hai

THEORY

Index ek special data structure hai (aksar **B-Tree** ya **Hash** based) jo table ke columns par bana kar search ko fast karta hai — bilkul kisi kitab ke aakhir mein diye gaye "index" ki tarah, jahan aap topic dhoondne ke liye पूरी kitab parhne ke bajaye seedha page number dekh lete hain.

Bina Index vs Index ke Sath

- **Bina Index:** Database ko **Full Table Scan** karna parta hai — matlab har row check karni parti hai. Time: $O(n)$.
- **Index ke sath:** B-Tree structure ki wajah se search $O(\log n)$ mein ho jati hai — bohat fast, especially large tables ke liye.

```
CREATE INDEX idx_employee_name ON employees(name);

-- Ab ye query fast hogi:
SELECT * FROM employees WHERE name = 'Ali';
```

Search with Index: $O(\log n)$ | Search without Index: $O(n)$ | Insert/Update/Delete with Index: thora slower ho jata hai (kyunke index bhi update karna parta hai)

Trade-off: Index search ko fast karta hai, lekin insert/update/delete ko thora slow karta hai (kyunke har baar index bhi update karna parta hai) aur extra storage space leta hai. Is liye har column par index nahi lagate — sirf un columns par jo frequently search/filter/join mein use hotay hain.

Types of Indexes

- **Primary Index:** Primary key par automatically ban jata hai.
- **Unique Index:** Ensure karta hai ke column mein duplicate values na hon.
- **Composite Index:** Multiple columns par ek sath index (jaise `(department_id, salary)`) — us waqt useful jab queries mein dono columns sath use hotay hon.

6. NoSQL Databases — Types & Use Cases

THEORY

NoSQL databases 4 main types mein aati hain, har ek ka apna use case hai:

1. Document Database

Data **JSON-like documents** ki form mein store hota hai. Har document flexible structure rakh sakta hai (schema-less).

```
{
  "id": 1,
  "name": "Ali",
  "skills": ["Python", "ML", "SQL"],
  "address": { "city": "Lahore", "country": "Pakistan" }
}
```

Examples: MongoDB, CouchDB. **Use case:** Content management, user profiles, catalogs — jahan data structure varying ho.

2. Key-Value Database

Sab se simple type — har entry ek **key** aur uski **value** hoti hai, dictionary/hashmap ki tarah. Bohat fast lookup deta hai.

Examples: Redis, DynamoDB. **Use case:** Caching, session storage, real-time leaderboards.

3. Column-Family Database

Data columns ke groups mein store hota hai (rows ke bajaye), jo bare, distributed datasets ke liye fast read/write deta hai.

Examples: Cassandra, HBase. **Use case:** Time-series data, large-scale logging, IoT data.

4. Graph Database

Data **nodes aur relationships (edges)** ki form mein store hota hai — connections/relationships khud data ka important part hotay hain.

Examples: Neo4j, Amazon Neptune. **Use case:** Social networks, recommendation engines, fraud detection.

5. Vector Database (AI Engineers ke liye Sab Se Important)

Ye specifically embeddings (high-dimensional vectors) store karne aur unpar **similarity search** karne ke liye design hui hain. Agla section isi par detail se focus karega, kyunke ye AI Engineer role ke liye sab se zyada relevant hai.

Examples: Pinecone, Weaviate, Chroma, Milvus, Qdrant, FAISS (library, standalone DB nahi).

7. Vector Databases

AI ENGINEER — MOST IMPORTANT

Vector Database ek aisi database hai jo data ko **embeddings** (numbers ke lambay arrays, jaise `[0.12, -0.45, 0.88, ...]`) ki form mein store karti hai, aur "meaning" ke hisab se similar items dhoondne deti hai — traditional keyword match ke bajaye.

Ye Kyun Zaroori Hai?

Traditional SQL database `WHERE name = 'Ali'` jaisi exact match queries deti hai. Lekin AI applications (jaise ChatGPT-type systems, semantic search, recommendation engines) ko "meaning ke hisab se similar" cheezein chahiye hoti hain — jaise "car" aur "automobile" alfaaz mein different hain lekin meaning mein same hain. Ye kaam sirf **vector similarity search** se ho sakta hai, traditional SQL se nahi.

Kaise Kaam Karta Hai — Step by Step

1. **Embedding banana:** Text/image/audio ko ek AI model (jaise OpenAI's `text-embedding-3`, ya `sentence-transformers`) se pass karke ek numeric vector banaya jata hai (jaise 1536 dimensions ka array). Similar meaning wali cheezein similar vectors banati hain.
2. **Store karna:** Ye vectors Vector Database mein store hotay hain, saath mein original text/metadata bhi.
3. **Query karna:** Jab user koi question puchta hai, uska bhi embedding banaya jata hai, aur database mein **closest/most similar vectors** dhoonde jate hain.

Simple Python Example (using FAISS — ek popular vector search library)

```
import faiss
import numpy as np

# Maan lein humare paas 3 documents ke embeddings hain (4-dimensional, example ke liye)
embeddings = np.array([
    [0.1, 0.2, 0.3, 0.4], # doc 1: "cat"
    [0.11, 0.19, 0.29, 0.41], # doc 2: "kitten" (cat se milta julta)
    [0.9, 0.8, 0.7, 0.6] # doc 3: "airplane" (bilkul different)
], dtype='float32')

index = faiss.IndexFlatL2(4) # 4 = dimensions
index.add(embeddings)

query = np.array([[0.1, 0.2, 0.3, 0.39]], dtype='float32') # "cat" jaisa query
distances, indices = index.search(query, k=2) # top 2 closest results
print(indices) # Output: [[0, 1]] -> doc 1 (cat) aur doc 2 (kitten) sab se close hain
```

Similarity Kaise Measure Hoti Hai

- **Cosine Similarity:** Do vectors ke beech "angle" measure karta hai (1 = same direction/very similar, 0 = unrelated, -1 = opposite). Text embeddings ke liye sab se common metric.
- **Euclidean Distance (L2):** Do points ke beech seedhi line ka distance. Jitna kam distance, utna zyada similar.
- **Dot Product:** Vectors ka multiplication sum — magnitude aur direction dono consider karta hai.

ANN — Approximate Nearest Neighbor Search

Agar aapke paas millions/billions vectors hon, to har query par **sab se** compare karna (exact/brute-force search) bohat slow hoga ($O(n)$). Is liye Vector DBs **ANN algorithms** use karte hain jo approximate (lekin bohat close) results bohat fast dete hain:

- **HNSW (Hierarchical Navigable Small World)**: Sab se popular ANN algorithm — ek "graph" based structure banata hai jisme similar vectors connected hotay hain, is se search $O(\log n)$ ke qareeb ho jati hai.
- **IVF (Inverted File Index)**: Vectors ko clusters mein group karta hai, query ke waqt sirf relevant clusters search karta hai (poora dataset nahi).
- **PQ (Product Quantization)**: Vectors ko compress karta hai taake memory kam use ho, thori accuracy ki qeemat par.

Brute Force (Exact) Search: $O(n)$ — sab vectors se compare | ANN Search (HNSW/IVF): approximately $O(\log n)$ — bohat fast, thori si accuracy trade-off ke sath

Popular Vector Databases — Comparison

Database	Type	Notes
Pinecone	Managed Cloud Service	Fully managed, easy setup, production-ready, paid
Weaviate	Open-source / Cloud	Built-in hybrid search (keyword + vector), GraphQL API
Chroma	Open-source (lightweight)	Local development ke liye best, RAG prototypes mein bohat use hoti hai
Milvus	Open-source	Large-scale production systems ke liye, highly scalable
Qdrant	Open-source / Cloud	Rust mein likhi, fast aur filtering support achi
FAISS	Library (Facebook/Meta)	Database nahi, sirf ek search library — khud persistence/scaling implement karni parti hai
pgvector	PostgreSQL Extension	Existing Postgres database mein hi vector search add kar deta hai

Interview mein yaad rakhein: "Vector Database = Embeddings store karta hai + ANN algorithm (jaise HNSW) se fast semantic/similarity search deta hai — traditional keyword match ke bajaye 'meaning' ke hisab se results deta hai."

8. Embeddings & Similarity Search — Concept

AI ENGINEER

Embedding basically kisi bhi cheez (text, image, audio) ki **numeric representation** hoti hai jo uska "meaning" capture karti hai. Ye ek neural network model (jaise BERT, OpenAI embeddings, CLIP) se generate hoti hai.

Zaroori Concept: Semantic Meaning Numbers Mein

Jab do sentences ka meaning similar hota hai, to unke embeddings bhi vector space mein ek dusre ke **qareeb** hotay hain. Jaise "Main bohat khush hoon" aur "I am very happy" — alag languages mein hain lekin agar model multilingual hai, to unke embeddings close hongay.

Python Example: Text ko Embedding Mein Convert Karna

```
# sentence-transformers library use kar ke (open-source)
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
sentences = ["Main AI Engineer hoon", "I work as an AI Engineer", "Aaj mausam acha hai"]
embeddings = model.encode(sentences)

print(embeddings.shape)  # (3, 384) -> 3 sentences, har ek 384-dimensional vector

# Cosine similarity check karna
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity([embeddings[0]], [embeddings[1]])
print(similarity)  # High value (close to 1) - kyunke dono ka meaning same hai
```

Chunking — Bare Documents ke liye Zaroori Step

Agar document bohat bara ho (jaise 50-page PDF), to usay ek hi embedding mein convert karna sahi nahi — meaning "dilute" ho jata hai. Is liye document ko chote **chunks** (jaise 200-500 words) mein tor kar, har chunk ka alag embedding banaya jata hai. Ye RAG systems ka bohat important step hai.

Chunking Strategy Interview Question: Chunk size bohat chota ho to context lost ho jata hai, bohat bara ho to irrelevant info bhi retrieve ho jati hai. Overlap (jaise 50 words overlap consecutive chunks mein) rakhna best practice hai taake context boundary par break na ho.

9. RAG mein Database ka Role

AI ENGINEER

RAG (Retrieval Augmented Generation) ek technique hai jisme LLM (jaise GPT) ko apna khud ka data/knowledge base "retrieve" karne diya jata hai, taake wo accurate, up-to-date, aur specific answers de sakay — sirf apni training data par rely kiye bagair.

| RAG Pipeline — Step by Step

1. **Data Preparation:** Documents ko load karein, chunks mein tor lein.
2. **Embedding:** Har chunk ka embedding banayein (embedding model se).
3. **Storage:** Embeddings ko Vector Database mein store karein (chunk text aur metadata ke sath).
4. **Query Time:** User ka question aye to uska bhi embedding banayein.
5. **Retrieval:** Vector DB mein similarity search karein — top-k (jaise top 5) sab se relevant chunks nikalein.
6. **Generation:** Ye retrieved chunks LLM ke prompt mein "context" ki tarah add karein, aur LLM se final answer generate karayein.

Simplified RAG Flow (Pseudo-code)

```
def rag_pipeline(user_question, vector_db, llm):  
    # Step 1: Query ka embedding banayein  
    query_embedding = embed_model.encode(user_question)  
  
    # Step 2: Vector DB se top relevant chunks nikalein  
    relevant_chunks = vector_db.similarity_search(query_embedding, k=5)  
  
    # Step 3: Context ke sath prompt banayein  
    context = "\n".join(relevant_chunks)  
    prompt = f"Context: {context}\n\nQuestion: {user_question}\nAnswer:"  
  
    # Step 4: LLM se answer generate karayein  
    answer = llm.generate(prompt)  
    return answer
```

Interview mein yaad rakhein: "RAG mein Vector Database ka role hai relevant context ko fast retrieve karna, taake LLM ko sirf apni training knowledge par nahi, balke actual/updated data par based answer dene mein help mile. Ye hallucination bhi kam karta hai."

10. Feature Store — ML Systems ke liye

AI ENGINEER

Feature Store ek special database/system hai jo ML models ke liye **features** (jaise "user_age", "average_purchase_amount", "last_login_days_ago") ko centrally store, manage, aur serve karta hai.

| Ye Kyun Zaroori Hai?

- **Training-Serving Skew Avoid Karna:** Bina Feature Store ke, training ke waqt features alag tareeqe se calculate hotay hain aur production/inference ke waqt alag — jo bugs ka bara source hai. Feature Store dono jagah **same** feature calculation guarantee karta hai.
- **Reusability:** Ek team ka banaya feature dusri team bhi use kar sakti hai, dobara calculate karne ki zaroorat nahi.
- **Fast Serving:** Real-time predictions ke waqt features milliseconds mein chahiye hotay hain — Feature Store isko optimize karta hai (aksar Redis jaisi key-value store use hoti hai serving layer ke liye).

Do Types ke Stores

- **Offline Store:** Bara historical data, model training ke liye (jaise data warehouse: BigQuery, Snowflake).
- **Online Store:** Fast, low-latency access, real-time inference ke liye (jaise Redis, DynamoDB).

Popular Tools: Feast (open-source), Tecton, Databricks Feature Store.

11. Caching & Scaling

THEORY

Caching (Redis)

Caching matlab frequently accessed data ko ek **fast, temporary storage** (usually RAM-based) mein rakhna, taake bar bar main database se query na karni paray — jo slow hoti hai.

```
import redis

r = redis.Redis(host='localhost', port=6379)

r.set('user:1:name', 'Ali', ex=3600) # 1 hour ke liye cache karein
name = r.get('user:1:name')          # Fast O(1) lookup, database hit nahi hua
```

AI Engineer Angle: LLM applications mein caching bohat use hoti hai — jaise same/similar questions ke embeddings ya LLM responses cache kar dena, taake repeat queries par API cost aur latency dono kam ho.

Scaling — Sharding & Replication

- **Replication:** Database ki multiple copies (replicas) rakhna. Ek "primary" (write) aur baaki "replicas" (read) — read-heavy systems ke liye load distribute karta hai aur agar primary fail ho jaye to backup available rehta hai.
- **Sharding (Horizontal Partitioning):** Data ko multiple servers mein tor kar rakhna (jaise user IDs 1-1000 server A par, 1001-2000 server B par). Bohat bara data handle karne ke liye zaroori hota hai jab ek server ki capacity kam par jaye.

Replication = Same data, multiple copies (reliability + read scaling) | Sharding = Different data, multiple servers (write scaling + capacity)

12. Bonus: Interview Questions & Cheat Sheet

BONUS

Common Interview Questions (Practice Karein)

- SQL vs NoSQL kab use karte hain? Example ke sath explain karein.
- ACID properties kya hain aur ye kyun zaroori hain?
- Index kaise database ko fast banata hai? Iska trade-off kya hai?
- Normalization aur denormalization mein farq, aur kab kya use karein?
- Vector Database traditional database se kaise different hai?
- RAG system design karein — Vector DB, embedding model, aur LLM kaise sath kaam karte hain?
- Cosine similarity aur Euclidean distance mein farq?
- Agar aapke paas 100 million embeddings hon, to exact search ki bajaye ANN kyun use karenge?
- Feature Store ka role ML pipeline mein kya hota hai?

Quick Cheat Sheet

Concept	One-Line Summary
RDBMS	Tables + relationships + fixed schema + ACID
NoSQL	Flexible schema, horizontal scaling, eventual consistency (BASE)
Index	B-Tree based structure, search $O(\log n)$ ke bajaye $O(n)$
ACID	Atomicity, Consistency, Isolation, Durability — reliable transactions
Vector DB	Embeddings store karta hai, ANN se semantic/similarity search deta hai
RAG	Retrieval (Vector DB) + Generation (LLM) = accurate, grounded answers
Feature Store	Centralized ML features, training-serving consistency
Caching (Redis)	Fast temporary storage, database load kam karta hai
Sharding	Data ko multiple servers mein split karna (capacity scaling)
Replication	Data ki multiple copies (reliability + read scaling)

All the best! Database concepts DSA jaisi practice nahi maangte, lekin clear samajh maangte hain — especially Vector Databases aur RAG, kyunke ye AI Engineer interviews mein tezi se important ho rahe hain. In concepts ko apne khud ke words mein explain karne ki practice karein, aur agar mumkin ho to ek chota RAG project khud bana kar dekhein (Chroma + sentence-transformers se local development free hai) — practical experience se confidence bohat barhata hai.